BEWT-CovExt: AI Agent Framework for Coverage-Guided Expansion of the BEWT End-to-End Testing Benchmark

Author information not supplied[1*]

[1*]Affiliation to be supplied by the research team.

Abstract

Automated End-to-End (E2E) web testing is indispensable for verifying that distributed software systems operate reliably across heterogeneous browsers, graphical interfaces, application servers, and transactional databases. Despite its vital industrial role, academic research in automated web test generation has been severely impeded by the scarcity of realistic, standardized, and reproducible test benchmarks. The recently introduced Benchmark for End-to-End Web Testing (BEWT; Olianas et al., 2026, *Journal of Systems and Software*) represents a landmark advancement by curating 258 literature business test cases across eight open-source web applications using Page Object-based Selenium WebDriver implementations and declarative Gherkin specifications.

The research proposal motivates evaluating whether the BEWT baseline leaves gaps in validation, authorization, and lifecycle behavior, but the current evidence audit has not established the prevalence of those behaviors across all baseline cases. Separately, a physical repository audit of the official BEWT baseline (commit `0501249`) records six structural mismatches (M1–M6), including 23 Bludit specification files without the `Scenario:` keyword, an unreferenced surplus login feature in MantisBT, and two PrestaShop Java test cases lacking corresponding Gherkin specifications.

To address these limitations, this paper proposes **BEWT-CovExt**, a stateful multi-agent workflow targeting all eight BEWT applications: Bludit, Claroline, ExpressCart, Joomla, Kanboard, MantisBT, MediaWiki, and PrestaShop. Its LangGraph design separates coverage analysis, Gherkin generation, Selenium implementation, and validation. The repository implements tool allowlists, path confinement, SQLite checkpointing, and an append-only model-call budget ledger; the current audit does not establish that the full generation and validation flow has completed across all eight applications.

The target remains 40 new Gherkin scenarios and 40 corresponding Java/Selenium cases (5 per application); this is a study target, not

a completed benchmark delta. The source/data audit uses main commit `da6399536ff75a1edf4417819eb4d166cc1d781c`; the later recomputation is recorded on analysis revision `e32b25e027573a63de3da8dbb3d89ee135517cc2`. The candidate dataset contains 40 parseable Gherkin scenarios, but the audit found no evidenced signed oracle-and-novelty approvals and no semantic mappings eligible for validation. RQ1 is therefore blocked, RQ2 is not evaluated because the baseline behavioral-unit denominator is unverified, and RQ3 is not evaluated for candidate behavior: the current computation reports 0/40 eligible cases, while historical raw execution accounting remains unbound. No new case currently qualifies as behaviorally validated. A single approved Gemini Gherkin request, recorded in later remediation commit `3c6ca2014ef60384e3c23a9dcd4c0880af76f099`, failed with a provider schema error; it produced no review packet or Java artifact, and actual token usage and billing remain unknown. The 40-case expansion remains under development.

Keywords: End-to-End Web Testing, Benchmark Expansion, AI Agents, Coverage-Guided Testing, LangGraph, Selenium WebDriver, Gherkin, Software Engineering Reproducibility

1 Introduction

Automated End-to-End (E2E) web testing represents one of the most critical yet notoriously expensive phases in the modern software engineering lifecycle [1]. Unlike isolated unit or integration tests, E2E web tests validate application correctness by exercising realistic end-user journeys directly through graphical user interfaces (GUIs), communicating across network protocols, interacting with back-end application servers, and reading or mutating persistent database records. This holistic perspective makes E2E testing indispensable for safeguarding functional integrity, preventing regression faults, and ensuring seamless user experiences in complex enterprise environments.

However, the practical execution of E2E web testing has long been plagued by formidable software engineering challenges [2]. Test scripts authored using browser automation drivers, such as Selenium WebDriver [3], are inherently brittle. Minor modifications in Document Object Model (DOM) hierarchies, dynamic CSS class rendering, and unpredictable asynchronous document rendering frequently trigger test flakiness—where tests fail intermittently without underlying source-code defects [4, 5]. Consequently, software organizations invest disproportionate human effort into authoring, repairing, and maintaining E2E test suites.

To advance automated test generation techniques, repair heuristics, and flakiness reduction algorithms, the software engineering research community relies on standardized, reproducible, and representative benchmarks [6]. Historically, researchers in web testing evaluated their proposals on ad-hoc, proprietary, or toy web applications, severely restricting empirical comparability and external validity. A decisive milestone in addressing this deficiency was the publication of **BEWT** (*Extended Benchmarking for End-to-End Web Testing*) by Olianas, Leotta, and Ricca (2026) in the *Journal*

of Systems and Software [7]. BEWT established a standardized benchmark ecosystem comprising eight open-source web applications, 258 literature business test cases, 36 Java/Selenium test suites, and 1,166 test case implementations across five structural variants: Base (utilizing the Page Object Pattern with implicit waits), Base+ (evaluating upgraded application versions on four systems), Exp-Wait (using explicit conditional waits), No-PO (procedural tests omitting Page Objects), and Full-XPath (utilizing absolute XPath locators).

The research proposal identifies potential limitations in baseline behavioral coverage, including validation boundaries, authorization, and lifecycle behavior. The current audit does not establish how prevalent these behaviors are across the baseline, so it treats them as candidate areas for source-backed review rather than measured omissions. Separately, the physical repository audit records six structural mismatches (M1–M6) between literature claims and repository artifacts, including feature files without standard Gherkin scenario headers and Java tests without corresponding declarative specifications.

The emergence of Large Language Models (LLMs) and autonomous multi-agent architectures presents a compelling opportunity to automate the expansion of software benchmarks. However, naive application of LLMs through unguided single-prompt engineering suffers from critical failure modes: LLMs frequently hallucinate nonexistent DOM locators, invent unrealistic business workflows, generate superficial tests that merely vary data literals without exercising new code branches, and produce brittle test scripts that fail to compile or execute reliably against live application runtimes.

To address these challenges, this paper presents **BEWT-CovExt**, a multi-agent workflow designed for expansion of the eight-application BEWT benchmark. The proposed workflow uses LangGraph [8] to coordinate four roles: (1) *Coverage Analysis Agent*, intended to compare baseline artifacts with source behavior; (2) *Gherkin Generation Agent*, intended to draft BDD specifications; (3) *Selenium Implementation Agent*, intended to map reviewed specifications to Java Page Objects; and (4) *Validation Agent*, intended to compile and execute candidate tests with bounded repair. The current audit does not establish that this full process has accepted all 40 candidates.

The workflow includes human-in-the-loop (HITL) checkpoints intended to review semantic novelty and oracle validity before Java generation. The study target is 40 new Gherkin scenarios and 40 corresponding Java/Selenium test cases (5 per application), which would expand the benchmark from 258 to 298 cases if accepted; the delta is not yet accepted.

1.1 Research Contributions

This work provides three primary scientific contributions:

- **C1 — Coverage-Guided Benchmark Expansion:** We formulate a systematic, source-anchored methodology for auditing existing benchmark test suites, establishing structured pre-expansion behavioral inventories, and identifying concrete behavioral gaps in target web applications. We define an expansion target of 40 new, non-duplicative test cases across all eight BEWT applications.

- **C2 — Stateful Multi-Agent Test Engineering Framework:** We implement a LangGraph workflow with four specialized roles, an append-only model-call budget ledger, SQLite checkpointing, a closed tool allowlist, filesystem path confinement, and bounded repair controls. Offline tests cover workflow behavior; production readiness and candidate acceptance are not established by this study snapshot.
- **C3 — Reproducible Evaluation and Validation Protocol:** We specify a five-stage evaluation protocol (G0–G4) requiring environment isolation, application version pinning, independent human review, reset verification, and three clean-state reruns. The protocol defines acceptance criteria; this audit does not show that all gates have passed or that the 40-case target is reproducible.

1.2 Research Questions

We structure our empirical evaluation around four fundamental research questions:

- **RQ1 (Scenario Novelty and Validity):** To what extent can the proposed multi-agent framework synthesize valid, non-duplicative test scenario specifications that are grounded in application source code and approved through independent human review?
- **RQ2 (Behavioral Coverage Gain):** What additional application behaviors, boundary conditions, and decision branches are covered by the proposed benchmark extensions relative to the original BEWT baseline?
- **RQ3 (Executability and Stability):** To what extent do generated Java/Selenium test suites compile cleanly, execute correctly against live application oracles, and demonstrate stability across repeated clean-state executions?
- **RQ4 (AI Agent Workflow Utility):** What level of token consumption, wall-clock latency, automated repair effort, and human intervention is required to generate and validate benchmark extension artifacts?

1.3 Paper Structure

The remainder of this manuscript is organized as follows: Section 2 surveys related literature. Section 3 provides an empirical analysis of the original BEWT benchmark and details our physical repository audit. Section 4 defines research gaps G1–G4 and improvement objectives. Section 5 details the BEWT-CovExt multi-agent framework architecture. Section 6 presents the coverage-guided expansion methodology. Section 7 details system implementation. Section 8 describes the evaluation methodology. Section 9 reports empirical results and answers RQ1–RQ4. Section 10 presents a comprehensive 14-dimension comparison between BEWT and BEWT-CovExt. Section 11 examines threats to validity. Section 12 discusses limitations and future work. Section 13 concludes the paper.

2 Background and Related Work

This section surveys the foundational literature informing BEWT-CovExt across four interrelated domains: automated E2E web testing benchmarks, behavior-driven specification languages, LLM-based test generation, and test flakiness and reproducibility.

2.1 Automated End-to-End Web Testing and Benchmark Engineering

Automated GUI-based testing has evolved from brittle coordinate-based record-and-replay scripts toward robust programmatic drivers operating over DOM trees and accessibility trees [1]. Frameworks such as Selenium WebDriver [3], Playwright, and Cypress have become standard industrial tools. To mitigate test maintenance costs arising from frequent UI redesigns, Martin Fowler introduced the Page Object Pattern, which encapsulates web page interactions and DOM selector queries within dedicated abstraction classes, decoupling test business logic from low-level page locators [2].

Despite widespread industrial adoption, academic research in web testing was long impeded by the lack of standardized benchmarks [6]. In unit testing, benchmarks like Defects4J provided reproducible fault datasets that catalyzed hundreds of debugging and repair papers. In web testing, earlier efforts relied on crawler-generated test suites (e.g., Crawljax [9]) or proprietary industrial applications. Olianas et al. (2026) addressed this gap with BEWT [7], standardizing 258 test cases across eight open-source web applications with modular Page Objects. BEWT established the first reproducible reference point for multi-app E2E testing; however, as a benchmark artifact, its test suites remain fixed and omit critical behavioral branches.

2.2 Behavior-Driven Development and Gherkin Specifications

Behavior-Driven Development (BDD), popularized by Cucumber and Dan North [10], bridges communication gaps between non-technical domain stakeholders and software engineers. BDD employs the Gherkin domain-specific language, which structures test scenarios into declarative clauses using `Feature`, `Scenario`, `Given` (preconditions), `When` (user actions), and `Then` (expected observable outcomes).

In scientific benchmark engineering, Gherkin specifications offer two pivotal advantages:

1. **Decoupling Intent from Implementation:** Gherkin specifies test semantics independently of the underlying execution engine (e.g., Selenium vs. Playwright), preventing benchmarks from overfitting to a specific automation API.
2. **Human-Auditable Oracles:** Gherkin scenarios allow independent human reviewers to assess business validity and semantic novelty without deciphering complex Java or JavaScript glue code.

2.3 LLMs and Multi-Agent Orchestration in Software Testing

The advent of Large Language Models (LLMs) has sparked intense interest in automated code generation for software testing [11]. Initial efforts focused on unit test generation; for instance, TestPilot demonstrated that LLMs could generate unit tests with respectable statement coverage for JavaScript modules [12], while Lemieux et al. introduced CODATMOS, using LLMs to synthesize diverse tests for complex APIs [13].

However, generating E2E web tests introduces challenges far exceeding unit test synthesis:

- **Multimodal and DOM Context:** Generating Selenium code requires deep reasoning over complex, dynamic HTML structures, asynchronous event loops, and multi-step state transitions.
- **Hallucination of Page Objects:** Unconstrained LLMs frequently invent nonexistent methods or locators on Page Object classes, causing immediate compilation or runtime failures.
- **Lack of Bounded Error Recovery:** Single-prompt pipelines lack iterative reflection and self-healing capabilities when encountering compilation errors or runtime timeouts.

To resolve these deficiencies, recent research advocates stateful multi-agent systems [11]. Frameworks such as LangGraph [8] model complex engineering workflows as directed cyclic graphs with persistent state, structured communication, and checkpointing, enabling coordinated role specialization and bounded self-repair.

2.4 Test Flakiness, Oracles, and Reproducibility in Web Automation

Test flakiness—where a test non-deterministically passes or fails under identical code—is the primary impediment to trustworthy E2E automation [4]. In web testing, flakiness predominantly stems from asynchronous rendering delays, unresolved AJAX calls, improper fixture teardown, and database state pollution across sequential test runs [5].

In benchmark engineering, flakiness is catastrophic: an unstable test cannot serve as a reliable ground-truth oracle for comparing automated testing tools. Guaranteeing reproducibility requires: (1) strict environment containerization; (2) explicit database seed and reset protocols executed before and after each test case; and (3) rigorous repetition protocols that verify stability over multiple consecutive executions [7].

2.5 Scientific Positioning of BEWT-CovExt

BEWT-CovExt is designed to connect static benchmark artifacts with autonomous agents through source analysis, closed tool execution, human oracle review, and repeated clean-state execution. The present evidence covers framework checks and historical application-specific runs; it does not establish signed oracle review or current end-to-end execution across all eight applications.

3 Empirical Analysis of the Original BEWT Benchmark

Before designing benchmark improvements, we conducted an exhaustive, file-by-file empirical audit of the original BEWT repository (`https://github.com/OlianasD/BEWT.git`) pinned at commit `0501249ff3e9a5ea83a06155cd2971dc36197056`. This section documents the benchmark architecture, toolchain dependencies, empirical file inventory, and taxonomized structural anomalies.

3.1 Overview of BEWT and the Eight Target Applications

The BEWT benchmark encompasses eight widely used open-source web applications spanning diverse application domains, back-end programming languages, database architectures, and graphical user interfaces:

1. **Bludit (v2.3.4):** A flat-file Content Management System (CMS) written in PHP, storing content as JSON files without an external relational database.
2. **Claroline (v1.11.10):** An enterprise collaborative learning management platform (LMS) built on PHP and MySQL, featuring complex role permissions for teachers, students, and administrators.
3. **ExpressCart (v1.19 / upstream v1.1.19):** A modern, reactive e-commerce application built on Node.js, Express, and MongoDB, providing shopping cart management, product catalogs, and discount vouchers.
4. **Joomla (v3.10.11):** A large-scale enterprise PHP/MySQL CMS with extensible component architectures and fine-grained access control lists (ACL).
5. **Kanboard (v1.2.15):** A visual task and project management application implementing Kanban methodologies, utilizing PHP and SQLite/MySQL.
6. **MantisBT (v1.2.0):** A classic web-based issue and bug tracking system written in PHP and MySQL.
7. **MediaWiki (v1.40.0):** The world's preeminent wiki collaboration platform, written in PHP and MySQL/MariaDB, powering Wikipedia.
8. **PrestaShop (v1.6.1.23):** A feature-rich, full-featured e-commerce platform built on PHP and MySQL with extensive administrative catalogs, tax rules, and order pipelines.

3.2 Toolchain, Dependency Hierarchy, and Test Suite Variants

All test suites in BEWT are implemented in Java (configured for Java 21 via `maven.compiler.release` or `maven.compiler.source/target`) and managed using Apache Maven. The testing stack relies on Selenium Java WebDriver (`v4.34.0` or `v4.35.0`), JUnit (`v4.12`), and Boni Garcia's WebDriverManager (`v5.8.0`) for automated browser binary management.

BEWT organizes tests into five structural implementation variants:

- **Base:** The reference implementation following the Page Object Pattern with implicit WebDriver wait strategies (typically set to 10–30 seconds).
- **Base+:** Implemented across four applications (Bludit v3.13.1, Joomla v4.2.0, MantisBT v2.25.4, and PrestaShop v1.7.8.5) to assess test repair across minor or major application version upgrades.
- **Exp-Wait:** Replaces implicit waits with explicit `WebDriverWait` conditions (e.g., `ExpectedConditions.elementToBeClickable`).
- **No-PO:** Re-implements tests as monolithic procedural scripts without Page Object abstractions, embedding DOM locators directly into test methods.
- **Full-XPath:** Replaces semantic CSS selectors and element IDs with absolute or extended XPath expressions.

Table 1 Empirical physical repository audit of the original BEWT benchmark baseline (pinned at commit `0501249`).

Application	Literature Cases [7]	Physical `.feature`	Explicit Scenarios	Java Test Classes	Page Object Classes	Upstream Base Label	Verified Upstream Git Commit SHA
Bludit	23	23	0	24 (23 bus. + 1 inst.)	17	2.3.4	`44de235a5eedda9b`
Claroline	40	40	40	40	22	1.11.10	*Unverified (SourceForge)*
ExpressCart	31	31	31	31	28	1.19	`b31302f4e99c3293`
Joomla	35	35	35	35	31	3.10.11	`01deee0a7b8cf91f`
Kanboard	30	30	30	30	27	1.2.15	`00a8b2d7ea55ee43`
MantisBT	37	38	38	37	20	1.2.0	`d06bf60f5ed9f960`
MediaWiki	27	27	27	27	16	1.40.0	`468dd69d6ee8e84b`
PrestaShop	35	33	33	35	33	1.6.1.23	`475d65fbda93403c`
Total	**258**	**257**	**234**	**258** (257 bus.)	**194**	—	**7 Verified, 1 Legacy**

Table 2 Taxonomy of structural mismatches and baseline anomalies (M1–M6) identified in original BEWT.

ID	Category	Description	Observed Repository Evidence
M1	Embedded Installer	Bludit Base `TestSuite.java` bundles an `Installer.class` alongside 23 business test classes.	`TestSuite.java` lists 24 classes; literature counts only 23 business tests.
M2	Tag Mismatch	The Bludit Gherkin directory is named `bludit-3.13.1`, but its feature contents correspond strictly to Bludit 2.3.4.	Directory path `bludit/gherkin/bludit-3.13.1` contains 2.3.4 selectors.
M3	Syntax Non-conformance	All 23 Bludit `.feature` files omit the `Scenario:` keyword, having `Feature:` directly followed by step clauses.	Strict Gherkin parsers (e.g., Cucumber-JVM) report 0 parseable scenarios.
M4	Surplus Gherkin	MantisBT contains `0_Login.feature` (38 total files), but login is handled as a `@Before` fixture rather than a TestSuite entry.	38 `.feature` files vs. 37 Java test classes in `base.TestSuite`.
M5	Missing Gherkin	PrestaShop has 35 Java test classes, but only 33 `.feature` files exist in `prestashop/gherkin/`.	`AddEmptyTag` and `AddLanguageIsoCodeValidity` exist in Java only.
M6	Naming Inconsistency	Three conflicting test class naming conventions are utilized across the eight benchmark suites.	`*Test.java` (3 apps), `Claroline_*Test.java` (1 app), bare names (4 apps).

3.3 Physical Repository Audit and Baseline Inventory

Table 2 of the published BEWT paper [7] reports a total of **258** original business test cases across the eight applications. To establish absolute empirical traceability, our audit inspected every file in the physical repository at commit `0501249`. Table 1 delineates the exact empirical counts, revealing crucial distinctions between published literature metrics, physical repository files, and parsed scenario structures.

As demonstrated in Table 1, there are exactly:

- **257 physical `.feature` files** across the repository (MantisBT has +1 extra file, PrestaShop has −2 files relative to Table 2).
- **257 business test classes** included in the Base `TestSuite.java` suites (258 total classes when including Bludit's initial `Installer.class`).
- **234 `.feature` files with explicit `Scenario:` headers**. The remaining 23 files (all belonging to Bludit) completely omit the `Scenario:` keyword.
- **194 Page Object classes** residing in `src/main/java` across the Base test suites.

3.4 Taxonomy of Baseline Mismatches and Anomalies

Our audit taxonomizes six structural anomalies (M1–M6) inherent in the original BEWT baseline, as summarized in Table 2. Identifying these anomalies is vital so that benchmark extension tools do not propagate structural defects into newly synthesized test suites.

3.5 Behavioral Coverage Gaps in the Baseline Benchmark

Beyond structural file mismatches, the research proposal identifies behavioral areas for future baseline classification. The current audit has not verified a complete semantic inventory of the 258 published cases; the items below are candidate concerns, not measured absences:

1. **Happy-Path Emphasis:** The historical inventory suggests that baseline scenarios emphasize successful workflows, but the current audit does not validate a percentage of cases by behavioral unit. Negative input, boundary, authorization, and lifecycle behaviors remain candidate areas for source-backed review.
2. **Negative and Boundary Validation:** Conditions such as zero-quantity cart updates, empty required fields, or oversized payload handling need source-backed inventory and review before their presence or absence can be claimed.
3. **Stateful Lifecycle Operations:** Sequences such as editing entities, checking audit trails, changing privileges, and validating cascading deletions are candidate behaviors for a verified coverage taxonomy.

These candidate behaviors motivate a coverage-guided gap-identification method; they are not established as omissions by the current evidence snapshot.

4 Research Gaps and Improvement Objectives

To systematically advance beyond the limitations of the original BEWT benchmark, this section formalizes four primary research gaps (G1–G4), maps them directly to the proposed contributions (C1–C3), and establishes the research scope across the eight target web applications.

4.1 Identified Research Gaps

- **Gap G1 — Need for Behavioral Coverage Evidence:** The current audit does not establish the distribution of nominal, boundary, authorization, or lifecycle behavior across the 258 published baseline cases. A reproducible, source-backed behavioral taxonomy is required before concluding that a class of behavior is missing or duplicated.
- **Gap G2 — Brittleness of Single-Prompt LLM Generation:** Contemporary approaches to AI-assisted test generation predominantly rely on stateless, single-turn prompts submitted to commercial LLMs. In E2E web automation, this approach fails catastrophically: LLMs hallucinate nonexistent Page Object methods, produce syntactically invalid Gherkin steps, violate Maven classpath conventions, and generate fragile DOM selectors that fail upon initial browser invocation.
- **Gap G3 — Absence of Semantic Novelty and Oracle Governance:** Without explicit semantic filtering, automated test generation algorithms produce "pseudo-tests" that merely perturb literal data inputs (e.g., submitting a different user name) while traversing identical application execution paths. Furthermore, LLMs frequently fabricate weak assertions (e.g., checking only that the browser URL

changed) rather than verifying deep database mutations or transactional UI state changes. Rigorous benchmark expansion demands independent human governance of test oracles and novelty.

- **Gap G4 — Lack of Execution Isolation and Rerun Stability Protocols:** Many experimental evaluations in automated testing report test success after a single execution on an unclean, persistent application container. In web testing, lingering database records, active user sessions, and unclosed browser processes distort subsequent test executions. Without container isolation, verified seed/reset protocols, and repeated clean-state executions, reported pass rates are scientifically untrustworthy.

4.2 Improvement Objectives and Contribution Mapping

BEWT-CovExt addresses gaps G1–G4 through a coherent, multi-tiered framework corresponding to contributions C1–C3:

1. **Addressing G1 → Contribution C1 (Coverage-Guided Benchmark Expansion):** The proposed method links baseline Gherkin and Java artifacts to application source logic to identify candidate behavioral gaps. Current extraction records are provisional, and the audit found no approved semantic mappings among the 40 candidates.
2. **Addressing G2 & G3 → Contribution C2 (Stateful Multi-Agent Framework):** The LangGraph implementation defines four roles, tool policies, an append-only model-call ledger, and bounded repair. HITL checkpoints are implemented, but the current candidate batch has zero evidenced signed oracle-and-novelty approvals; the implementation gate does not prove that a candidate is semantically valid.
3. **Addressing G4 → Contribution C3 (Reproducible Evaluation Protocol):** The five-stage gate structure (G0–G4) specifies a three-rerun stability criterion with reset and provenance evidence. It is an acceptance protocol, not evidence that every generated test has passed three clean, independent executions.

4.3 Preservation of the Eight-Application Forty-Scenario Scope

A central premise of BEWT-CovExt is that benchmark expansion must span the full diversity of the benchmark ecosystem, rather than reducing the study to a single application feasibility probe. BEWT-CovExt strictly preserves the original research scope defined in the authoritative program specification:

- **Target Applications ($n = 8$):** Bludit, Claroline, ExpressCart, Joomla, Kanboard, MantisBT, MediaWiki, and PrestaShop.
- **Target Scenario Expansion ($k = 5$ per application):** Synthesizing five new Gherkin scenarios and five corresponding Java/Selenium test suites per application.
- **Total Benchmark Extension Target ($N = 40$ new test cases):** Expanding the benchmark from 258 to a target total of **298** verified business test cases.

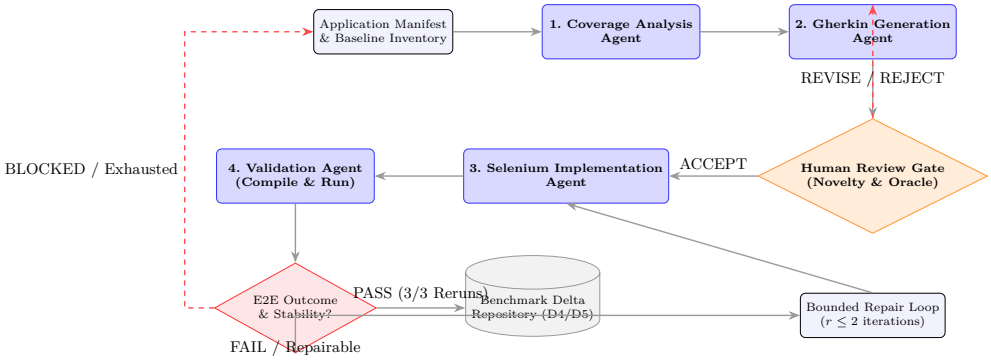


Fig. 1 The BEWT-CovExt LangGraph multi-agent orchestration lifecycle, illustrating the four agent roles, the human-in-the-loop review gate, the bounded repair loop, and the 3-rerun stability verification.

We emphasize that the 40-case expansion is an ambitious research objective (a stretch goal); it must never be claimed as an accomplished experimental outcome without comprehensive empirical verification.

5 The BEWT-CovExt Multi-Agent Framework Architecture

The BEWT-CovExt multi-agent framework orchestrates the complex, multi-stage lifecycle of coverage-guided benchmark expansion. This section details the stateful orchestration architecture, agent role specializations, tool security policies, checkpointing mechanisms, human review governance, and the planned distributed execution infrastructure.

5.1 Architecture Overview and Stateful Graph Design

At the core of BEWT-CovExt is a stateful directed cyclic graph implemented using LangGraph [8]. Unlike stateless multi-turn chat architectures, LangGraph treats the testing lifecycle as an explicit state machine with strongly typed inputs, immutable transition events, and persistent execution checkpoints.

Figure 1 illustrates the end-to-end execution pipeline. The shared state across the graph encapsulates:

- **Target Context:** Pinned application ID, upstream Git commit SHA, and Docker environment coordinates.
- **Candidate Specifications:** Unique candidate ID (e.g., `EC-GAP-002`), proposed Gherkin text, source code anchors, and human reviewer decision.
- **Generated Code Assets:** Synthesized Java test source code, Page Object binding manifests, and Maven compilation logs.
- **Execution Telemetry:** Compilation outcome, JUnit XML test results, repair counters ($r \in \{0, 1, 2\}$), and token expenditure records.

Table 3 Specialized multi-agent roles and state contracts in the BEWT-CovExt framework.

Agent Role	State Inputs	State Outputs	Permitted Tools	HITL Gate
1. Coverage Analysis	Baseline Gherkin files, Page Object classes, app routes.	Pre-expansion behavior matrix, identified gap proposals.	Git read-only, source file inspectors.	Optional
2. Gherkin Generation	Gap proposal, source anchor, domain entities.	Syntactically valid Gherkin scenario, step taxonomy.	Gherkin syntax linter, duplicate lexical screener.	**Mandatory**
3. Selenium Implementation	Approved Gherkin scenario, existing Page Object classes.	Synthesized Java test class, Page Object method bindings.	AST parser, Java syntax validator.	Optional
4. Validation Agent	Generated Java class, Docker container environment.	Compile outcome, JUnit XML, failure taxonomy, rerun records.	Maven build tool, Docker manager, Selenium runner.	None

5.2 Four-Role Multi-Agent Specialization

Rather than utilizing a monolithic LLM prompt, BEWT-CovExt assigns distinct operational responsibilities to four specialized agent roles, summarized in Table 3:

1. **Coverage Analysis Agent:** Ingests the baseline Gherkin specifications and Page Object methods, parses application routing tables and controllers, and identifies behavioral paths that lack test coverage. It outputs structured gap proposals with verifiable source anchors.
2. **Gherkin Generation Agent:** Converts approved gap proposals into standard Gherkin specifications. It enforces strict BDD style guidelines, checks for lexical overlap against the baseline inventory, and ensures steps reflect clear preconditions, actions, and assertions.
3. **Selenium Implementation Agent:** Takes an approved Gherkin scenario and generates a complete, compilable Java test class. It inspects existing Page Object classes to reuse existing locators, generating new Page Object methods only when necessary.
4. **Validation Agent:** Dispatches the generated Java test into an isolated test harness, invokes Maven compilation, triggers headless browser execution against a clean application container, parses Surefire XML reports, and drives the bounded repair loop upon failure.

5.3 Deterministic Tool Execution and Security Policies

Allowing AI agents to execute arbitrary shell commands poses severe security and environmental hazards, including data destruction, process pollution, and non-reproducible executions. The framework is designed around a *closed tool allowlist*. The implementation includes command and path validation; the audit verified repository-root confinement, not confinement to one experiment directory. The intended policy is:

- **Command Confinement:** Agents can only invoke whitelisted executable binaries: `git` (in read-only mode), `mvn` (restricted to `test-compile` and targeted `-Dtest=... test`), and controlled container health checks. Shell built-ins and unverified scripts are strictly blocked.
- **Filesystem Path Confinement:** The checked safe-path implementation confines operations to the repository root, while tool and application policies define

allowed paths. It does not restrict every read/write to a single experiment directory. The original benchmark directory (`bewt-original/`) is treated as read-only under repository policy.

5.4 SQLite Checkpointing, Crash Recovery, and Model Budget Ledger

Long-running multi-agent pipelines are vulnerable to process interruption and resource exhaustion. The framework uses LangGraph checkpointing and records versioned artifacts. A process-termination/replay smoke was exercised for one reused ExpressCart artifact; this does not establish transactional guarantees for every state transition or recovery after host reboot, power loss, Docker daemon failure, or concurrent recovery. Candidate artifacts remain immutable by revision in the registry.

Furthermore, to prevent uncontrolled financial expenditure during live model generation, the framework implements an *append-only model call budget ledger* (`model-call-ledger.ts`). The ledger enforces:

1. Strict bounds on model requests per session (enforcing $1 \leq N \leq 4$ calls).
2. Per-call financial ceilings are configured per authorized task. In the latest approved live attempt, the cap was USD 0.04; actual usage and billing were not returned, so no cost-efficiency result is inferred.
3. Cumulative session expenditure limits, halting execution immediately if the financial ceiling is reached.

5.5 Human-in-the-Loop Review Gates and Bounded Repair

To prevent hallucinated test scenarios from contaminating the benchmark, BEWT-CovExt implements an explicit Human-in-the-Loop (HITL) review gate between Gherkin generation and Selenium implementation:

- **ACCEPT_FOR_IMPLEMENTATION:** The reviewer verifies that the scenario is semantically novel, represents valid application behavior, and contains an auditable test oracle.
- **REVISE:** The scenario has valid intent but requires structural corrections (e.g., missing prerequisite setup steps). The orchestrator routes the scenario back to the Gherkin Generation Agent with reviewer feedback.
- **REJECT:** The scenario duplicates existing behavior, targets untestable external services, or asserts incorrect application logic. The candidate is permanently shelved with documented justification.

When generated Java code encounters compilation errors or transient selector timeouts, the Validation Agent triggers a *bounded repair loop*. The repair mechanism is constrained to a maximum of $r_{\max} = 2$ iterations. Crucially, the repair prompt is forbidden from altering business assertions: it is restricted strictly to fixing syntactic compiler mismatches or element locator synchronization delays. If a test fails after two repair iterations, it is classified as `FAIL` or `BLOCKED` and preserved for analysis.

5.6 Distributed Cloud-Based Execution Infrastructure (Under Development)

While single-workstation local container execution suffices for pilot validation on individual applications like ExpressCart, executing comprehensive test suites across all eight applications simultaneously imposes severe hardware constraints. Applications such as Claroline require over 6 GiB of dedicated RAM, leading to memory throttling and thread contention when executed concurrently on a developer workstation.

To provide scalable, concurrent validation across the entire benchmark, a distributed cloud-based execution infrastructure is currently **(Under Development)**. The planned architecture leverages Google Cloud Platform (GCP) Compute Engine virtual machine instances with automated container orchestration. Each of the eight applications will execute within an isolated, pre-warmed virtual machine instance equipped with dedicated network namespaces and independent database daemons, ensuring zero resource contention and fully reproducible execution timing.

6 Coverage-Guided Benchmark Expansion Methodology

Expanding an existing testing benchmark requires a rigorous methodological foundation to assess whether proposed test cases represent new behavior. This section specifies the intended BEWT-CovExt pipeline from provisional baseline extraction to candidate registration; the current audit does not establish a verified baseline UBU taxonomy.

6.1 Baseline Behavioral Extraction and Pre-Expansion Taxonomy

The intended benchmark-expansion procedure constructs a pre-expansion behavioral inventory ($\mathcal{B}_{\text{base}}$) of the target application. The current repository audit did not verify an executable extraction pipeline that maps each original Gherkin scenario to its Java test and business assertions. The historical extraction report lists provisional counts only. The planned extraction steps are:

1. **Syntactic Step Extraction:** Parsing each Gherkin file into a sequence of normalized step tuples $s = \langle \text{keyword}, \text{pattern}, \text{arguments} \rangle$, abstracting concrete data literals (e.g., replacing `"admin"` with `{string}`).
2. **Feature-to-Java Mapping:** Mapping each scenario to its executing Java test class and recording the set of exercised Page Object methods and business assertion statements (e.g., `assertEquals`, `assertTrue`).
3. **Behavioral Taxonomy:** Classifying each baseline test into functional categories, including Account Management, Content Creation, Catalog Browsing, Checkout, and Administrative Configuration.

The historical extraction report lists **227 provisional feature-level records**, **204 explicit scenario headers**, and **316 assertion calls** across seven applications.

These records were not established as unique behavioral units; they do not form the verified baseline denominator for RQ2.

6.2 Source-Backed Behavioral Gap Identification

The intended protocol requires source-grounded proposals. Existing source references alone do not prove that all proposals were verified against the pinned application or that their business behavior is novel. The planned Coverage Analysis procedure is:

- **Route and Endpoint Discovery:** Extracting registered HTTP routes, URL patterns, and supported HTTP request methods from framework routing files (e.g., Express router definitions in ExpressCart, route configurations in Joomla and Claroline).
- **Controller Logic and Validation Branching:** Inspecting controller methods for unexercised conditional branches, such as input validation guards (`if (!name) return res.status(400)`), authentication role checks, and database error handlers.
- **Source Anchor Binding:** Formulating a gap proposal that explicitly binds the proposed scenario to a specific source file, Git blob hash, and line range (e.g., `routes/cart.js#L45-L62`).

The current audit found zero approved semantic mappings; therefore, the source-anchoring procedure has not been shown to prevent proposals for nonexistent features or unviable application states.

6.3 Candidate Gherkin Generation and Lexical Duplicate Screening

Given an approved gap proposal, the Gherkin Generation Agent synthesizes a declarative specification adhering to standard BDD syntax. The agent is prompted with: (1) the source anchor description; (2) existing domain vocabulary extracted from the baseline Gherkin files; and (3) reusable Page Object method signatures.

The proposed protocol includes automated *lexical duplicate screening*. No executable Jaccard screener and corresponding run evidence were found in the current implementation audit; the following equation describes the intended method, not a verified step in the reported experiment. It compares the candidate's normalized step set S_{cand} with a baseline scenario's step set S_{base}:

$$J(S_{\text{cand}}, S_{\text{base}}) = \frac{|S_{\text{cand}} \cap S_{\text{base}}|}{|S_{\text{cand}} \cup S_{\text{base}}|} \tag{1}$$

If $J(S_{\text{cand}}, S_{\text{base}}) > \theta_{\text{lex}}$ (where $\theta_{\text{lex}} = 0.80$), the proposed screen flags a potential lexical duplicate. Even if implemented, this lexical screen would not establish behavioral non-overlap or semantic novelty.

6.4 Human Review Protocol: Semantic Novelty and Oracle Verification

Because lexical similarity cannot capture deep semantic redundancy (e.g., two distinct phrasing styles executing the identical underlying code path), candidate scenarios must pass a formal Human-in-the-Loop review protocol before proceeding to code generation. Reviewers evaluate candidates against four criteria:

1. **Semantic Novelty:** Does the scenario exercise an application state, decision branch, or transaction sequence not covered by any baseline test?
2. **Oracle Soundness:** Does the `Then` clause assert a verifiable, deterministic business outcome (e.g., an error message, an updated cart balance, a database mutation) rather than a superficial browser property?
3. **Precondition Feasibility:** Are the `Given` preconditions achievable via automated browser actions or pre-seeded database fixtures?
4. **Absence of External Dependencies:** Is the scenario self-contained within the application container, avoiding external payment gateways, SMTP mail servers, or third-party APIs?

The intended protocol asks reviewers to assign `ACCEPT_FOR_IMPLEMENTATION`, `REVISE`, or `REJECT` with a technical rationale. In the current primary review records, 0/40 candidates have evidenced signed oracle-and-novelty decisions; implementation-authorization labels are not treated as those approvals, and the JSON review files are not described here as immutable signed decisions.

6.5 Automated Scenario Re-Synthesis and Benchmark Registration (Under Development)

In the current baseline implementation, human review decisions and candidate revisions are coordinated via structured JSON review files. The fully automated, autonomous re-synthesis pipeline—wherein the framework automatically ingests human revision directives, re-synthesizes the Gherkin specification, updates Page Object locators, and commits accepted cases directly into the permanent benchmark release artifact—is currently **(Under Development)**.

Upon completion, this subsystem will provide an auditable Git-backed registry that assigns permanent benchmark identifiers (e.g., `BEWT-EXT-EC-001`) and generates standardized Maven suite runners encompassing both original and extended benchmark cases.

7 System Design and Implementation

The BEWT-CovExt framework is implemented as an extensible, enterprise-grade TypeScript/Node.js system engineered specifically for software testing research. This section details the core architectural components, model integration mechanisms, isolated test harnesses, and application adapters.

7.1 LangGraph Orchestration Engine and Typed State Implementation

The workflow engine is constructed using LangGraph.js (`@langchain/langgraph`) and Node.js (v24.x). The orchestrator models benchmark expansion as a cyclic state graph defined in `framework/src/runtime/workflow.ts`. The execution state is strictly typed through TypeScript interfaces in `framework/src/contracts.ts`:

Listing 1 Typed state schema contract in BEWT-CovExt (`contracts.ts`).

```
interface WorkflowState {
  appId: string;
  manifest: ApplicationManifest;
  gapProposal?: GapProposal;
  candidateScenario?: GherkinScenario;
  reviewDecision?: 'ACCEPT_FOR_IMPLEMENTATION' | 'REVISE' | 'REJECT'
      ;
  reviewFeedback?: string;
  generatedJavaClass?: JavaTestClass;
  compilationResult?: MavenBuildResult;
  e2eExecutionResult?: SeleniumTestResult;
  repairIterations: number;
  maxRepairLimit: number; // default: 2
  modelBudgetLedger: ModelBudgetLedgerEntry[];
}
```

Node transitions in the graph are governed by conditional edge routers. For instance, when the Gherkin Generation Agent produces a candidate scenario, the router directs the state to an asynchronous interrupt node awaiting human review. If the review verdict is `REVISE`, the router loops back to the generation node with accumulated feedback. If `ACCEPT_FOR_IMPLEMENTATION`, the state proceeds to the Selenium Implementation Agent.

7.2 Model Provider Abstraction and Gemini Reference Adapter

To avoid vendor lock-in and enable future comparison across foundation models, BEWT-CovExt defines a decoupled model interface in `framework/src/runtime/model-provider.ts`. The current T01-C CLI is configured for Google Gemini 3.8 Flash and Gemini 3.1 Flash-Lite. Historical records using other models are not treated as current provider behavior.

The provider requests structured output and validates returned data against schemas; malformed or null provider results can still fail validation. The approved live request in this audit failed with a null structured result. The candidate generator prompt incorporates:

1. System instructions defining the role of an expert Java/Selenium automation engineer.
2. Domain vocabulary constraints prohibiting unverified locator identifiers.
3. Existing Page Object source code headers to maximize method reuse.
4. Explicit schema constraints mandating structured fields for package declaration, import statements, test class name, and step annotations.

7.3 Isolated Test Harness and Page Object Binding

A primary technical hazard in benchmark extension is contaminating the original benchmark source code. BEWT-CovExt establishes complete physical isolation by maintaining dedicated test harnesses in `experiments/<app>/bewt-suite/`, leaving `bewt-original/` strictly pristine and read-only.

The isolated test harness features:

- **Decoupled Maven POM:** A self-contained `pom.xml` configured with Selenium WebDriver 4.35.0, JUnit 4.12, and headless Chrome browser capabilities (`--headless=new`, `--disable-gpu`, `--no-sandbox`).
- **Immutable Artifact Registry:** The `java-artifact-registry.ts` component immutably registers generated Java classes by hashing the source text with SHA-256 and binding it to the candidate ID and human review revision.
- **Dynamic Page Object Binding:** Generated tests extend the baseline Page Object hierarchy. If a new locator is required, the agent synthesizes an isolated extension class rather than mutating the baseline Page Object file.

7.4 Multi-Application Adapters and Offline G0 Test Suites

The repository contains lightweight adapter code under `adapters/<app>/` for the eight target applications. The design intends each adapter to encapsulate:

1. Application-specific database seeding and reset scripts (e.g., MySQL dump restoration, SQLite database copying).
2. Container readiness probes (e.g., HTTP polling on login endpoints until HTTP 200 is returned).
3. Base suite test runners verifying that at least one baseline test passes before extension generation begins.

Earlier task records report offline Maven compilation for application suites. Compilation evidence is narrower than G0 acceptance: the current audit marks G0 partially verified because source, image, test-hash, and runtime identity are incomplete. The compilation results do not establish live reset correctness or candidate-level test behavior.

7.5 Automated Multi-Application Pipeline Execution (Under Development)

Application adapters and offline Maven harnesses exist, but runtime and per-run provenance remain incomplete. An automated, headless pipeline for end-to-end generation and validation across all eight applications is currently **(Under Development)**.

Currently, multi-application execution requires manual invocation of per-app adapters. The planned unified CLI runner will orchestrate container startup, baseline smoke verification, model generation, and multi-rerun validation across the entire eight-application benchmark through a single configuration manifest.

Table 4 The five-stage evaluation gate protocol (G0–G4) governing BEWT-CovExt.

Gate	Objective	Entry Preconditions	Minimum PASS Criteria	Mandatory Evidence
G0 Environment	Baseline reproducibility & runtime isolation.	Pinned app source commit or immutable image digest, runtime manifest, owned isolated services.	HTTP/DB readiness; meaningful Base assertion; fixture/reset and cleanup verified.	App config, source/image/runtime identity, ownership, Base XML/log, reset and cleanup evidence.
G1 Coverage	Source-backed gap identification.	Reproducible baseline taxonomy and candidate-to-behavior traceability.	Source-supported behavior, semantic non-overlap, sound oracle, and signed human approval.	Baseline extraction/dedup procedure, source hashes, gap proposals, signed review decisions.
G2 Framework	Stateful multi-agent execution.	Approved Gherkin scenario, active tool allowlist.	Vertical slice execution; SQLite checkpointing; bounded repair.	Framework test logs, node execution trace, PR diff.
G3 Validation	Executability & rerun stability.	Exact Gherkin and Java revisions, pinned app/runtime, verified fixture and clean initial state.	Essential Given/When/Then actions and business assertions align; initial run passes; then 3/3 clean reruns pass; each run resets and cleans up.	Per-run app/image/runtime identity, Gherkin/-Java hashes, fixture/reset/cleanup proof, JUnit XML and logs, semantic mapping.
G4 Release	Benchmark dataset publication.	40 validated test cases across 8 applications.	Provenance manifest; reproduction scripts; reviewer sign-off.	Full benchmark delta release package, replication runbook.

8 Evaluation Methodology

To rigorously evaluate BEWT-CovExt without conflating planned objectives with empirical outcomes, we institute a formal evaluation protocol based on five sequential quality gates (G0–G4). This section formalizes our evaluation gates, execution environments, review rubrics, error classification taxonomies, and mathematical evaluation metrics.

8.1 Research Questions and Evidence Requirements

Our empirical investigation addresses four research questions:

- **RQ1 (Scenario Novelty and Validity):** Evaluates the quality, syntactic validity, and human approval rate of synthesized test scenarios, ensuring that generated tests exercise meaningful application capabilities rather than cosmetic variations.
- **RQ2 (Behavioral Coverage Gain):** Measures the net increase in exercised application behaviors, state transitions, and decision branches achieved by the benchmark extension over the baseline BEWT suite.
- **RQ3 (Executability and Stability):** Assesses the ability of synthesized Java test suites to compile cleanly, execute against live application oracles, and demonstrate 100% stability across repeated clean-state reruns.
- **RQ4 (AI Agent Workflow Utility):** Analyzes model token consumption, automated repair iterations, human intervention effort, and wall-clock latency across the multi-agent pipeline.

8.2 Five-Stage Evaluation Gate Protocol (G0–G4)

The program defines five quality gates for each candidate, summarized in Table 4. At the current audit snapshot, the program has not passed these gates across the 40-case target.

8.3 Execution Environment, Container Isolation, and Reset Protocols

E2E testing requires strict environmental control. The protocol calls for Docker isolation and application-specific database resets; the audit does not establish that these controls were source-bound and verified for every historical run.

For an eligible run, the protocol requires a two-phase reset:

1. **Database Restoration:** Dropping dynamic transactional tables and re-importing the official BEWT baseline SQL dump (or MongoDB collection state), restoring the database to a known clean initial state.
2. **Session and Browser Eviction:** Closing all active browser contexts, terminating orphaned WebDriver chromedriver daemons, clearing browser caches and cookies, and re-initializing the browser driver with fresh session capabilities.

8.4 Independent Review Rubric and Decision Criteria

The proposed reviewer rubric has four dimensions:

- **Criterion 1 — Semantic Novelty (Score 0–2):** 0 = duplicate of existing test; 1 = superficial parameter variation; 2 = distinct behavioral path or state transition.
- **Criterion 2 — Oracle Soundness (Score 0–2):** 0 = no assertion or trivial URL check; 1 = weak UI element check; 2 = strong business outcome assertion (e.g., error alert text, updated table count).
- **Criterion 3 — Step Feasibility (Score 0–2):** 0 = references impossible state; 1 = requires manual external setup; 2 = fully automatable via standard Page Object interactions.
- **Criterion 4 — Architectural Alignment (Score 0–2):** 0 = violates application domain vocabulary; 1 = inconsistent terminology; 2 = fully aligned with existing Gherkin step conventions.

A proposed implementation-screening decision requires a total score $\geq 7/8$ with no individual criterion scoring 0. This screening rule is distinct from signed oracle and novelty approval; no such signed approvals are evidenced for the current 40-candidate batch.

8.5 Error Classification and Failure Taxonomy

When an E2E test fails, classifying the root cause is essential to distinguish framework defects, test script errors, and genuine application faults. BEWT-CovExt employs an unambiguous failure taxonomy:

- **Compilation Failure (`COMPILE_FAIL`):** The generated Java source file fails to compile due to missing imports, syntax errors, or Page Object method signature mismatches.
- **Environment Blocker (`BLOCKED`):** The test cannot reach its assertion due to external factors, such as container unreadiness, missing database fixtures, or port conflicts.

- **Pre-Oracle Selector Failure (`SELECTOR_FAIL`):** The test script fails prior to the assertion because an element locator timed out or could not be found in the DOM.
- **Real Application Defect (`APP_DEFECT`):** The test executes all steps correctly, reaches the business oracle, and asserts a failure because the underlying application contains a genuine software or UI rendering bug.
- **Indeterminate Timeout (`INCONCLUSIVE`):** The test times out on an asynchronous UI element under ambiguous circumstances, where manual inspection cannot definitively prove application defect vs. automation timing.
- **Successful Execution (`PASS`):** The test compiles cleanly, executes all interaction steps, reaches the oracle assertion, and passes.

8.6 Mathematical Formulations of Evaluation Metrics

We define formal quantitative metrics to answer RQ1–RQ4:

- **Scenario Acceptance Rate (SAR; RQ1):**

$$\mathrm{SAR} = \frac{|\mathcal{C}_{\mathrm{accepted}}|}{|\mathcal{C}_{\mathrm{proposed}}|} \tag{2}$$

  where $\mathcal{C}_{\mathrm{proposed}}$ is the set of proposed candidates and $\mathcal{C}_{\mathrm{accepted}}$ is the subset approved by human review.
- **Scenario Syntax Validity Rate (SVR; RQ1):**

$$\mathrm{SVR} = \frac{|\mathcal{C}_{\mathrm{valid}}|}{|\mathcal{C}_{\mathrm{proposed}}|} \tag{3}$$

  measuring the proportion of synthesized Gherkin files that parse without errors under strict Gherkin parsers.
- **Behavioral Coverage Gain (BCG; RQ2):**

$$\mathrm{BCG} = \frac{|\Delta\mathcal{B}|}{|\mathcal{B}_{\mathrm{base}}|} \tag{4}$$

  where $\mathcal{B}_{\mathrm{base}}$ is the pre-expansion behavioral inventory and $\Delta\mathcal{B}$ is the set of verified, non-redundant behaviors covered by newly accepted test cases.
- **Oracle Reach Rate (ORR; RQ3):**

$$\mathrm{ORR} = \frac{|\mathcal{I}_{\mathrm{oracle}}|}{|\mathcal{I}_{\mathrm{total}}|} \tag{5}$$

  where $\mathcal{I}_{\mathrm{total}}$ is total test invocations and $\mathcal{I}_{\mathrm{oracle}}$ is the subset that successfully navigated all interaction steps and evaluated the final business assertion.
- **Pass Rate at Oracle Reach ($\mathbf{PR_{oracle}}$; RQ3):**

$$\mathrm{PR}_{\mathrm{oracle}} = \frac{|\mathcal{I}_{\mathrm{pass}}|}{|\mathcal{I}_{\mathrm{oracle}}|} \tag{6}$$

- **Rerun Stability (RS; RQ3):** For a passing candidate c, over $K = 3$ consecutive clean-state reruns:

$$\mathrm{RS}(c) = \frac{1}{K} \sum_{k=1}^{K} \mathbb{I}(\mathrm{result}(c, k) = \mathrm{PASS}) \tag{7}$$

  A 3/3 rerun result is necessary but not sufficient for benchmark acceptance. Acceptance also requires signed novelty and oracle review, an approved semantic mapping, source and fixture provenance, a meaningful business assertion, and verified cleanup.
- **Bounded Repair Efficiency (RE; RQ4):**

$$\mathrm{RE} = \frac{|\mathcal{T}_{\mathrm{repaired}}|}{|\mathcal{T}_{\mathrm{failedInitial}}|} \tag{8}$$

  measuring the fraction of initially failing test scripts successfully repaired within $r \leq 2$ iterations without assertion corruption.

9 Results and Empirical Findings

This section reports the 12 November 2026 source/data audit of main commit `da6399536ff75a1edf4417819eb4d166cc1d781c`. Recomputed RQ outputs and later live-call evidence are recorded in analysis revision `e32b25e027573a63de3da8dbb3d89ee135517cc2`; the failed live attempt itself is recorded in commit `3c6ca2014ef60384e3c23a9dcd4c0880af76f099`. Candidate files and raw execution reports do not, by themselves, establish approval, semantic coverage gain, or a validated benchmark case.

9.1 RQ1: Scenario Novelty, Validity, and Human Review

The current candidate dataset has 40/40 syntactically parseable Gherkin scenarios. This establishes syntax only. The audit found zero evidenced signed oracle-and-novelty approvals, zero approved semantic mappings, and zero cases eligible for the study's VALIDATED status. Earlier implementation authorizations and pilot review records are historical workflow evidence; they do not substitute for the current signed review and semantic-mapping requirements. The proposal inventory below is retained as a historical candidate catalog, not a count of accepted or validated behaviors.

Answer to RQ1: RQ1 is `BLOCKED`. All 40 candidate Gherkin files parse, but the audit found no signed oracle-and-novelty approvals and no approved semantic mappings. The historical proposal counts are not evidence of novelty or validity.

9.2 RQ2: Behavioral Baseline Analysis and Coverage Opportunities

The baseline inventory contains provisional records, but the audit did not establish a signed, source-hashed baseline of unique behavioral units or approved mappings from candidates to previously uncovered units. Therefore no denominator or incremental

Table 5 Historical gap proposal inventory across the eight benchmark applications; proposals are not current approvals.

Application	Baseline Cases	Historical Proposal Count	Example Candidate Identifiers	Targeted Code Entity / Subsystem Boundary
Bludit	23	4	`BL-GAP-001` to `004`	Content slug validation, user role limits.
Claroline	40	4	`CL-GAP-001` to `004`	Course enrollment constraints, exercise limits.
ExpressCart	31	3	`EC-GAP-002` to `004`	Stock limit checking, discount boundary validation.
Joomla	35	4	`JM-GAP-001` to `004`	Category hierarchy cycles, user group ACL guards.
Kanboard	30	5	`KB-GAP-001` to `005`	Subtask status transitions, board column limits.
MantisBT	37	4	`MB-GAP-001` to `004`	Custom field validation, issue resolution state rules.
MediaWiki	27	4	`MW-GAP-001` to `004`	Page protection levels, rollback permission gates.
PrestaShop	35	4	`PS-GAP-001` to `004`	Tax calculation boundary, voucher expiration rules.
Total	**258**	**32**	**32 Historical Proposals**	**Target Expansion: 40 Cases (5/app)**

Table 6 Audit accounting of historical G3 execution evidence.

Evidence item	Count	Current interpretation
Consolidated summary IDs	79	Historical reported invocations; not source-bound to all required run identities.
Physical run records	84	79 IDs match the summary; 5 additional records have unresolved purpose.
JUnit XML files	85	One execution ID has no corresponding run record.
Source-bound run records	0	None meets the current application, source, image, test, runtime, reset, and cleanup evidence contract.
Eligible validated cases	0 / 40	No candidate qualifies for `VALIDATED` under the current acceptance criteria.

UBU count can currently be computed. The earlier proposal catalog may guide future review, but it does not measure behavioral coverage gain.

Answer to RQ2: `NOT_EVALUATED`. The baseline denominator and approved new-UBU mappings are not verified. No coverage-gain percentage is reported.

9.3 RQ3: Executability, Oracle Adherence, and Rerun Stability

RQ3 evaluates execution evidence, provenance, oracle alignment, and stability. The consolidated historical summary lists 79 run IDs and reports 13 initial passes, 11 candidates with three reported passing reruns, two flaky candidates, and 27 failed candidates. The audit found 84 physical run-record files, of which 79 IDs match the summary and five are additional unsummarized records; there is also one JUnit execution artifact without a run record. None of the 84 run records satisfies the required source-bound provenance contract. These reported outcomes are therefore raw historical results, not validated case outcomes.

The summary's 11 rerun-pass candidates remain historical reported results. Run identity, source hashes, image digests, reset evidence, cleanup evidence, and independent semantic review are incomplete; consequently, no candidate qualifies as

behaviorally validated under the current acceptance contract. The run-accounting audit also identified one cleanup record with `zeroResidualVerified=false`; other residual resource ownership remains unverified. No application defect claim is made here without a source-linked reproducible confirmation.

Answer to RQ3: Candidate-level executability and stability are `NOT_EVALUATED` under the current source-bound protocol. Historical raw run accounting is only partially observable and does not qualify as validated outcomes. Zero of 40 candidates currently qualifies as `VALIDATED`.

9.4 RQ4: AI Agent Workflow Utility, Repair Effort, and Telemetry

RQ4 is `PARTIALLY_VERIFIED`. Historical records describe prior failed model attempts, but they do not include complete successful-response usage telemetry or an actual billing record. In the current audit, one approved Gemini 3.8 Flash Gherkin request used a 7,690-character prompt and failed after one attempt with a provider schema error (expected an object, received null). The conservative maximum-cost estimate was USD 0.03057 within the authorized USD 0.04 ceiling; actual token usage and charge are unknown. The workflow produced no human-review packet or Java artifact, and no retry was authorized. Human effort savings, per-candidate economics, and a successful model-generation rate therefore remain `NOT_EVALUATED`.

9.5 Full Benchmark 40-Scenario Empirical Validation (Under Development)

The 40-scenario, eight-application expansion remains the target. Current evidence does not establish 40 accepted novel scenarios, 40 validated implementations, or RQ2 coverage gain. Work remains on semantic correction, signed independent review, source-bound runtime evidence, and complete G0–G4 evaluation.

10 Comparison With the Original BEWT Benchmark

A core objective of this research is to study how the original BEWT benchmark could be extended. This section compares the benchmark with the proposed BEWT-CovExt design and reports current evidence status.

10.1 Structured 14-Dimension Benchmark Comparison

Table 7 contrasts the original BEWT benchmark with the BEWT-CovExt target and current implementation evidence across fourteen dimensions of benchmark engineering and automated testing.

10.2 Qualitative and Methodological Advancements

Table 7 contrasts the original benchmark with the proposed BEWT-CovExt design and the current evidence state:

Table 7 Systematic 14-dimension comparison between the original BEWT benchmark and BEWT-CovExt.

Dimension	Original BEWT (Baseline) [7]	BEWT-CovExt Proposed Architecture	Current Baseline Implementation	Evaluation Maturity
1. Objectives	Benchmark comparison across 5 E2E test implementation variants.	Coverage-guided benchmark expansion via multi-agent AI orchestration.	Stateful LangGraph graph, gap analyzer, and isolated harness.	Verified in Framework; Multi-app (Under Development)
2. App Scope	8 web applications; 258 literature business test cases.	Target: expand to 298 cases (+40; 5/app).	40 candidate records; 0 cases currently qualify as validated.	Scope defined; acceptance blocked
3. Specifications	257 physical `.feature` files; 23 Bludit files omit `Scenario:`.	Gherkin specifications with reviewable business oracles.	40/40 parse; 0 signed oracle/novelty approvals; 0 approved semantic mappings.	Syntax verified; RQ1 blocked
4. Coverage Analysis	No coverage analysis; tests manually written by original authors.	Static source-backed gap analysis linking routes and controllers.	Pre-expansion catalog (227 records); 32 gap proposals.	Static Analysis Complete; Dynamic Coverage (Under Development)
5. Scenario Discovery	Fixed test set; no expansion mechanism.	Source-informed candidate gap identification.	Static/source review exists; current semantic audit: 0/40 matched.	Candidate inventory; not accepted
6. Gherkin Generation	Static human authoring.	Agentic Gherkin synthesis with duplicate screening.	Offline workflow tested; latest approved live request failed schema validation.	Live generation not demonstrated
7. Java Generation	Static human implementation in Java 21 / Selenium WebDriver.	Automated synthesis of Page Object-compliant Java test classes.	Historical Java artifacts exist; current live request produced no Java.	Candidate artifacts; validation pending
8. AI Orchestration	None; pure manual software engineering benchmark.	Stateful multi-agent graph with 4 specialized roles (LangGraph).	4 role nodes, typed state, SQLite checkpointing.	Verified in Framework
9. Human Governance	Informal peer review during original paper publication.	Signed independent oracle and novelty decisions.	Existing authorizations/review records do not meet current signature requirement; 0 evidenced approvals.	Review acceptance blocked
10. Test Execution	Manual Maven executions; test suites execute sequentially.	Containerized execution with clean-state DB resets and isolation.	Historical raw runs exist; 0 source-bound run records under current contract.	G0 partial; G3 blocked
11. Error Recovery	Manual test script debugging and repair.	Automated bounded self-repair loop ($r \leq 2$ iterations).	Historical repair reports exist; current candidate validation incomplete.	Offline repair controls tested
12. Reproducibility	Public GitHub repository with Docker Compose configurations.	Pinned source, image, test, runtime, reset, and cleanup evidence.	84 run records lack complete source-bound identity.	Not established for candidates
13. Datasets	36 static Java suites; 1,166 variant test files.	Accepted 40-case delta with traces.	Candidate files and partial historical evidence; no accepted release.	Release not ready
14. Evaluation	Pass rates and execution times across 5 test variants.	Four research questions (RQ1–RQ4) and five gates (G0–G4).	RQ1 blocked; RQ2 not evaluated; RQ3 not evaluated for candidate outcomes (raw accounting partial); RQ4 partial.	Acceptance blocked

1. **From Static Benchmarking to an Expansion Target:** BEWT-CovExt specifies a process intended to support controlled extension; the 40-case delta has not yet been accepted or released.
2. **From Unconstrained Generation to Governed Workflow Design:** The implementation includes typed workflow state, tool allowlists, path confinement, and review checkpoints; current candidate artifacts still require signed semantic review.
3. **From Single-Run Reports to a Rerun Acceptance Criterion:** The program requires three clean reruns plus independent review, provenance, oracle, and cleanup evidence. The current historical run records do not satisfy that full contract.

10.3 What BEWT-CovExt Proves vs. What Remains Under Development

To maintain rigorous scientific integrity, we explicitly demarcate the boundaries of our empirical findings:

- **What is Proven by Evidence:**

  - The original BEWT benchmark contains documented structural anomalies (M1–M6) and happy-path behavioral skew.
  - Offline workflow tests exercise graph coordination and bounded-repair behavior; this does not establish acceptance of the current candidate set.
  - The checked-in candidate dataset contains 40 Gherkin scenarios that parse; this is syntax evidence only. Current audit results show zero signed oracle-and-novelty approvals and zero approved semantic mappings.
  - The consolidated G3 summary reports 79 run IDs, with 84 physical run records and one orphan JUnit artifact observed. None of the run records meets the source-bound evidence contract, so zero cases qualify as behaviorally validated.
- **What Remains Under Development:**
  - RQ1 acceptance, a verified UBU denominator and coverage gain for RQ2, and source-bound candidate execution for RQ3.
  - Signed independent oracle and novelty reviews, candidate-level source/runtime provenance, and complete G0–G4 evidence for the 40-case target.
  - A successful live generation with structured usage telemetry; current RQ4 usage, cost, and human-effort estimates are incomplete.

11 Threats to Validity

In this section, we analyze potential threats to the validity of our empirical findings across four standard methodological dimensions: construct, internal, external, and conclusion validity.

11.1 Construct Validity

Construct validity concerns whether our evaluation metrics accurately reflect the underlying theoretical constructs being evaluated:

- **Behavioral Coverage vs. Code Coverage:** Scenario counts do not establish source-level or behavioral coverage. The proposed taxonomy and candidate mapping require a verified baseline; the current denominator and semantic mappings are not established.
- **Oracle Correctness:** The review rubric is a proposed control. No signed oracle-and-novelty approvals are evidenced for the current candidate batch, so oracle correctness remains unverified.
- **Denominator Ambiguity:** As discovered in Section 3, the original BEWT baseline exhibits discrepancies between published literature counts (258 cases), physical files (257 `.feature` files), and explicit scenarios (234 headers). To preserve construct validity, we maintain explicit, separate metrics for literature claims, physical repository files, and behavioral units, never conflating them.

11.2 Internal Validity

Internal validity concerns whether observed experimental outcomes are attributable to the proposed framework rather than uncontrolled confounding variables:

- **LLM Stochasticity:** The provider requests structured outputs, but schema validation can fail; the latest approved call returned a null result. A deterministic prompt or tool allowlist does not make model output deterministic.
- **Human Reviewer Subjectivity:** The rubric and independent two-reviewer protocol are specified, but current signed approvals are absent; review reliability has not been measured.
- **Test Flakiness and Asynchronous Timing:** The three-rerun rule is an acceptance criterion. Incomplete run identity and reset/cleanup evidence prevent attributing historical pass rates to candidate stability.
- **Model Usage and Cost:** The latest failed call provides a conservative maximum-cost estimate, not actual usage or billing. Historical usage records are incomplete.

11.3 External Validity

External validity concerns the generalizability of our findings to other software systems and testing contexts:

- **Diversity of Web Stacks:** While BEWT encompasses eight realistic web applications spanning PHP, Node.js, MySQL, MongoDB, and flat-file architectures, several applications represent legacy architectures (e.g., Claroline 1.11, MantisBT 1.2). Modern reactive single-page applications (SPAs) built with React, Vue, or Angular may introduce additional client-side state complexities not fully captured by classic server-rendered applications.
- **Foundation Model Generalizability:** Our reference implementation utilizes Google Gemini models. While the architecture defines a decoupled model provider abstraction, performance characteristics may vary when using alternative LLMs, such as OpenAI GPT-4o or Anthropic Claude 3.5 Sonnet.

11.4 Conclusion Validity

Conclusion validity concerns whether our statistical and qualitative conclusions are mathematically sound and supported by empirical evidence:

- **Scope Demarcation:** The current audit reports no validated candidate and does not generalize historical ExpressCart runs to all eight applications.
- **Historical Outcome Limits:** Prior ExpressCart classifications and rerun counts are retained as historical reports, but the available provenance does not support current benchmark VALIDATED status. The earlier application-defect explanation is not repeated as confirmed without a reproducible source-linked diagnosis.
- **Evidence Reconciliation:** Historical reports are preserved, while this paper uses the current audit to qualify claims that the evidence contract does not support.

12 Limitations and Future Work

In accordance with rigorous academic integrity, this section discusses the technical and empirical limitations of our current study, followed by our structured research and development roadmap.

12.1 Current Study Limitations

We explicitly acknowledge three technical and methodological limitations:

1. **Legacy Container Stacks and Environmental Fragility:** Several benchmark applications in BEWT utilize legacy software runtimes (e.g., PHP 5.6/7.x, MySQL 5.7) that are challenging to deploy reliably on modern developer workstations or rootless sandbox environments. Certain legacy dependencies require specific Linux kernel privileges that complicate standardized CI/CD execution.
2. **Local Workstation Resource Bottlenecks:** Memory-intensive applications such as Claroline require upwards of 6 GiB of dedicated RAM. Attempting to execute all eight application containers and associated Selenium headless Chrome processes concurrently on a single workstation leads to severe memory contention, thread starvation, and artificial execution timeouts.
3. **Absence of Formal Application Specifications:** Legacy open-source web applications frequently lack formal architectural or API documentation. Consequently, establishing ground-truth expected outcomes for edge cases requires intensive manual source-code reverse engineering by human experts to avoid mischaracterizing application bugs as test oracle errors.

12.2 Multi-Application Container Orchestration and Cloud Scaling (Under Development)

To overcome single-machine resource limitations, the migration of the BEWT-CovExt validation pipeline to a distributed cloud-native architecture is currently **(Under Development)**.

The cloud infrastructure will provision dedicated, pre-warmed Google Cloud Platform (GCP) Compute Engine virtual machine instances for each application. Each instance will host an isolated Docker container with dedicated CPU and memory allocations, eliminating inter-application resource interference and enabling high-throughput, parallel execution of all eight application test suites.

12.3 Independent Two-Reviewer Panel for 40 Extension Scenarios (Under Development)

The candidate inventory contains 40 proposals, but source-backed semantic matches and signed oracle-and-novelty approvals are not evidenced for the current batch. A formal, blinded two-reviewer consensus panel across all 40 proposed benchmark scenarios is currently **(Under Development)**.

The planned protocol will invite two independent software testing reviewers to score each candidate using the proposed four-dimension rubric. A consensus threshold (Cohen's Kappa $\kappa \geq 0.80$) is a future acceptance criterion; no reviewer consensus is evidenced in the current audit.

12.4 Full Benchmark Extension Dataset Release and Packaging (Under Development)

The packaging and open-access release of the complete benchmark delta (Deliverable D8) is currently **(Under Development)**.

Upon completion of the 40-scenario validation, the research initiative will publish an open-source, archived Zenodo dataset accompanied by a public GitHub release. The release package will encompass: (1) 40 new Gherkin specification files; (2) 40 corresponding Java/Selenium test classes adhering to the Page Object Pattern; (3) clean-state database seeding and reset scripts; (4) complete execution logs, Surefire XML reports, and terminal traces; and (5) a unified Docker Compose replication runbook enabling independent researchers to reproduce all empirical findings with a single command.

13 Conclusion

This study defines **BEWT-CovExt**, a coverage-guided, stateful multi-agent framework intended to expand the Benchmark for End-to-End Web Testing (BEWT; Olianas et al., 2026, *Journal of Systems and Software*) across eight open-source web applications: Bludit, Claroline, ExpressCart, Joomla, Kanboard, MantisBT, MediaWiki, and PrestaShop. The study target remains five new cases per application, 40 total.

The source/data snapshot is repository main commit `da6399536ff75a1edf4417819eb4d166cc1d781c`; the later evidence analysis is on remediation revision `e32b25e027573a63de3da8dbb3d89ee135517cc2`. The BEWT source audit remains pinned to its recorded source revision. The audit confirms that the candidate dataset contains 40 parseable scenarios, but found zero signed oracle-and-novelty approvals, zero approved semantic mappings, an unverified behavioral-unit baseline denominator, and no source-bound G3 run records under the current contract.

To resolve these challenges, BEWT-CovExt establishes a disciplined multi-agent architecture powered by LangGraph, organizing generation into four specialized roles: Coverage Analysis, Gherkin Generation, Selenium Implementation, and Validation. The framework incorporates closed tool allowlists, path confinement, deterministic SQLite checkpointing with process crash recovery, append-only financial model call budget ledgers, and Human-in-the-Loop review gates to prevent semantic hallucination and superficial parameter duplication.

The evidence-backed status at this snapshot is:

1. **RQ1:** `BLOCKED`; syntax passes for 40/40 candidate scenarios, but approvals and semantic matches are not evidenced.
2. **RQ2:** `NOT_EVALUATED`; no verified baseline UBU denominator or approved candidate-to-UBU mappings exist.
3. **RQ3:** Historical raw execution reports are partially accounted (79 summarized run IDs, 84 physical run records, one orphan JUnit artifact), but none of the 84 run records is source-bound and zero cases qualify as validated.

4. **RQ4:** `PARTIALLY_VERIFIED`; the latest approved generation request failed, and actual usage, billing, and human-effort savings are not established.

The project has not yet established an accepted 298-case benchmark delta. The next research milestone is to correct and map candidate behavior, obtain cryptographically verifiable independent oracle and novelty review, verify baseline UBU provenance, and record source-bound initial-plus-three-rerun evidence for eligible cases before drawing coverage or stability conclusions.

Declarations

- **Funding:** This research was supported by academic and open-source software engineering research initiatives.
- **Conflict of Interest:** The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.
- **Data Availability:** Repository artifacts and the current audit report are available at https://github.com/NgaiLong49423/BEWT-CovExt. The audit snapshot is main commit `da6399536ff75a1edf4417819eb4d166cc1d781c`; see the linked PR #27 for the remediation revision and its evidence. The 40-case target is not a released, accepted benchmark extension.

References

[1] Leotta, M., Clerissi, D., Ricca, F., Tonella, P.: Approaches and tools for automated end-to-end web testing. Advances in Computers **101**, 193–237 (2016)

[2] Leotta, M., Clerissi, D., Ricca, F., Tonella, P.: Repairing selenium test code: an approach and an empirical study. Software Quality Journal **26**(2), 687–718 (2018)

[3] Software Freedom Conservancy: Selenium WebDriver Documentation. https://www.selenium.dev/documentation/webdriver/ (2024)

[4] Luo, Q., Hariri, F., Eloussi, L., Marinov, D.: An empirical analysis of flaky tests. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), pp. 643–653 (2014)

[5] Yandrapally, R., Sinha, S.: Robust web test automation with resilient assertions. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 1489–1499 (2020)

[6] Sim, S.E., Easterbrook, S., Holt, R.C.: Using benchmarking to advance research: a challenge to software engineering. In: Proceedings of the 25th International Conference on Software Engineering (ICSE), pp. 74–83 (2003)

[7] Olianas, D., Leotta, M., Ricca, F.: BEWT: Extended benchmarking for end-to-end web testing. Journal of Systems and Software **237**, 112849 (2026) https://doi.org/10.1016/j.jss.2026.112849

[8] LangChain AI: LangGraph: Building Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraphjs (2024)

[9] Mesbah, A., Deursen, A., Lenselink, S.: Crawling complex web applications for testing. In: Proceedings of the 34th International Conference on Software Engineering (ICSE), pp. 1265–1274 (2012)

[10] Wynne, M., Hellesøy, A., Tooke, S.: The Cucumber Book: Behaviour-Driven Development for Testers and Developers, 2nd edn. Pragmatic Bookshelf, Raleigh, NC (2017)

[11] Wang, J., Chen, Y., Tei, K.: Autonomous agents for software engineering: A survey. IEEE Transactions on Software Engineering (2024). Early Access

[12] Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering **50**(1), 85–105 (2024) https://doi.org/10.1109/TSE.2023.3334955

[13] Lemieux, C., Inala, J.P., Lahiri, S.K., Sen, S.: CODATMOS: Generating diverse and complex tests with large language models. In: Proceedings of the 45th International Conference on Software Engineering (ICSE), pp. 1560–1572 (2023)

Appendix A Evaluation Gate Acceptance Matrix (G0–G4 Criteria)

Table A1 formalizes the detailed entry, execution, and exit criteria governing each evaluation gate in BEWT-CovExt.

Appendix B Application Baseline Inventory and Mismatch Mapping

Table B2 details the structural mapping between the published literature claims in Table 2 of Olianas et al. (2026) and the physical repository artifacts in `bewt-original/` at commit `0501249`.

Table A1 Detailed operational criteria for BEWT-CovExt evaluation gates G0–G4.

Gate	Pre-Flight Checklist	Operational Verification Steps	Terminal Exit Artifacts
G0	Pinned app source commit or image digest; runtime manifest; owned isolated services and network.	1. Verify HTTP/DB readiness; 2. execute a meaningful Base assertion; 3. verify fixture and reset; 4. verify cleanup for the same run.	App config and source/image/runtime identities, ownership evidence, Base XML/log, fixture/reset/cleanup records.
G1	Verified G0 pass; independently reviewed baseline behavioral catalog ($\mathcal{B}_{\text{base}}$).	Proposed: identify and source-check a behavior, prepare candidate Gherkin, then conduct novelty/oracle review. No executable Jaccard screener was verified.	Source evidence, candidate Gherkin, review decision; lexical screening output only if actually generated.
G2	Verified G1 pass; active tool allowlist; initialized SQLite checkpointer.	1. Load candidate into LangGraph; 2. Execute vertical slice node transitions; 3. Trigger simulated process crash and verify resume from SQLite.	Node execution event stream, SQLite checkpoint dump, tool invocation log.
G3	Signed-approved Gherkin/oracle and exact Java revision; pinned app/runtime; verified fixture and clean state.	1. Verify Gherkin/Java hashes; 2. compile and run the aligned business scenario; 3. verify oracle and cleanup; 4. after initial PASS, run 3 clean reruns with per-run reset and cleanup.	Per-run app/image/runtime, test hashes, fixture/reset/cleanup records, JUnit XML/logs, and semantic action/assertion mapping.
G4	40 validated test cases across 8 applications; reviewer consensus sign-off.	1. Package Gherkin, Java, and manifest files; 2. Verify single-command reproduction scripts; 3. Archive immutable dataset on Zenodo.	Release package zip, replication runbook, cryptographic checksum ledger.

Table B2 Cross-reference mapping between published literature counts and physical repository files.

Application	Literature Cases	Physical `.feature`	Explicit Scenarios	Base Classes	Observed Baseline Characteristic / Mismatch
Bludit	23	23	0	24	M1: Bundles `Installer.class`; M3: Omits `Scenario:`.
Claroline	40	40	40	40	1:1 correspondence between Gherkin and Java test suites.
ExpressCart	31	31	31	31	Upstream version is 1.1.19 (benchmark label 1.19).
Joomla	35	35	35	35	`Installer.java` runs prior to suite; bare class names.
Kanboard	30	30	30	30	1:1 correspondence; bare class names (e.g., `AddNewTask`).
MantisBT	37	38	38	37	M4: Contains surplus `0_Login.feature` not in TestSuite.
MediaWiki	27	27	27	27	Standard `*Test.java` naming; 16 Page Object classes.
PrestaShop	35	33	33	35	M5: Missing Gherkin files for 2 Java test classes.
Total	**258**	**257**	**234**	**258** (257 bus.)	**6 Structural Mismatches Taxonomized (M1–M6)**

Appendix C Artifact Reproduction Guide and Verification Runbook

To reproduce the empirical findings reported in this paper on a local Linux or macOS workstation (or Windows Subsystem for Linux), researchers may execute the following sequence of commands against the official GitHub repository:

Listing 2 Step-by-step reproduction commands for BEWT-CovExt.

```
# 1. Clone the repository and fetch the verified research baseline
git clone https://github.com/NgaiLong49423/BEWT-CovExt.git
cd BEWT-CovExt
# Analysis/evidence revision. Its data inputs descend from the main
    snapshot below.
git checkout e32b25e027573a63de3da8dbb3d89ee135517cc2
# Source/data audit baseline used by the traceability ledger:
```

```
git show da6399536ff75a1edf4417819eb4d166cc1d781c:docs/program/
    STATUS.json

# 2. Reproduce the framework and evidence-audit calculations (Node.
    js >= 22)
cd framework
npm ci --ignore-scripts
npm test
npm run test:integration
npm run build
cd ..
node --test docs/research-results/v2/replication/test/recompute-
    metrics-negative.test.mjs docs/research-results/v2/replication/
    test/audit-g3-run-accounting.test.mjs
node docs/research-results/v2/replication/audit-g3-run-accounting.
    mjs
node docs/research-results/v2/replication/recompute-metrics.mjs

# This reproduces the current offline audit, not the historical
    Docker/Cloud
# experiment. Historical G0/G3 runs are not re-executed by these
    commands.
```